

Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
ISIS4826 – Robótica Móvil

Proyecto #3

Planificación Computacional de Caminos
Geométricos, Ejecución (Modo Autónomo)
– Continuación –

<https://github.com/Robotica-202610-a-cifci/proyecto3.git>

Profesor: Fernando De la Rosa

27 de mayo de 2026

Índice

1. Método de Planificación	2
1.1. Descripción del método computacional	2
1.1.1. Espacio de configuraciones	2
1.1.2. Algoritmo RRT	2
1.1.3. Post-procesamiento: suavizado y segmentación	3
1.2. Resultado del método de planificación	3
1.3. Traducción del camino en comandos al robot	3
2. Resultados en Simulación	4
2.1. Tabla de resultados – Método A* (Proyecto 2)	4
2.2. Tabla de resultados – Método RRT (Proyecto 3)	4
3. Análisis Comparativo	6
3.1. Diferencias conceptuales entre los métodos	6
3.2. Comparación de distancia lineal recorrida	6
3.3. Comparación de suma absoluta angular	7
3.4. Comparación de tiempo de generación	7
3.5. Comparación de tiempo de ejecución	8
3.6. Ventajas y limitaciones de cada método	8
4. Videos Ilustrativos	9
A. Estructura del Proyecto ROS2	9

1. Método de Planificación

1.1. Descripción del método computacional

El método de planificación implementado en este proyecto es el **RRT** (*Rapidly-exploring Random Tree*), un algoritmo probabilístico por muestreo que construye un árbol de exploración en el espacio de configuraciones continuo sin requerir una cuadrícula que restrinja las direcciones de desplazamiento del robot.

1.1.1. Espacio de configuraciones

A diferencia del proyecto 2, donde el C-espacio se discretizó en celdas de $\Delta = 0,25$ m, el RRT muestrea directamente en el espacio continuo $[0, W] \times [0, H]$ ($W = 4,0$ m, $H = 5,0$ m para los seis escenarios de prueba).

El robot se modela como un círculo de radio $r = 0,28$ m (**ROBOT_RADIO**). Cada obstáculo rectangular se *infla* por ese mismo radio, reduciendo la comprobación de colisión a verificar si el centro del robot entra en el rectángulo inflado. Las paredes reciben el mismo tratamiento: un punto (x, y) es libre si y solo si

$$r \leq x \leq W - r, \quad r \leq y \leq H - r,$$

y no cae dentro de ningún rectángulo inflado.

1.1.2. Algoritmo RRT

El algoritmo construye un árbol enraizado en q_0 expandiéndolo iterativamente hacia puntos muestreados al azar:

1. **Muestreo con sesgo hacia el objetivo:** con probabilidad $p_{\text{goal}} = 0,10$ se toma directamente q_f ; en caso contrario se muestrea uniformemente en el espacio libre.
2. **Nodo más cercano:** se busca en el árbol el nodo q_{near} con menor distancia euclídea al punto muestreado.
3. **Expansión:** se genera $q_{\text{new}} = \text{STEER}(q_{\text{near}}, q_{\text{sample}}, \delta)$ con paso $\delta = 0,30$ m (**STEP_SIZE**).
4. **Verificación de colisión:** se comprueba que tanto q_{new} como el segmento $\overline{q_{\text{near}} q_{\text{new}}}$ sean libres (muestreo denso a pasos de $r/2$).
5. **Condición de parada:** si $\|q_{\text{new}} - q_f\| < r_{\text{goal}} = 0,25$ m y el segmento directo hacia q_f es libre, se añade q_f al árbol y se reconstruye el camino por retroceso de padres.

El número máximo de iteraciones es **MAX_ITER** = 8 000.

Listing 1: Núcleo del algoritmo RRT (`planner.py`).

```
1 nodes = [q0_xy]; parents = [-1]
2 for _ in range(MAX_ITER):
3     sample = qf if rand() < GOAL_BIAS else uniform_sample()
```

```

4     near_idx = nearest(nodes, sample)
5     new_node = steer(nodes[near_idx], sample, STEP_SIZE)
6     if punto_libre(new_node) and segmento_libre(near, new_node):
7         nodes.append(new_node); parents.append(near_idx)
8         if dist(new_node, qf) < GOAL_RADIUS:
9             if segmento_libre(new_node, qf):
10                 nodes.append(qf); parents.append(len(nodes)-2)
11                 return reconstruir_camino(nodes, parents)
12 return None # sin solucion en MAX_ITER iteraciones

```

1.1.3. Post-procesamiento: suavizado y segmentación

Una vez obtenida la secuencia de nodos se aplican dos etapas:

1. **Suavizado (*line-of-sight*)**: se eliminan nodos intermedios redundantes comprobando visibilidad directa entre pares no consecutivos. Esto reduce drásticamente el número de giros.
2. **Segmentación de tramos largos**: todo segmento de longitud mayor a $\ell_{\max} = 1,0$ m se subdivide en trozos iguales, limitando la deriva odométrica acumulada.

1.2. Resultado del método de planificación

El resultado es una lista ordenada de *waypoints* (x_i, y_i, θ_i) en metros y grados que describe el camino geométrico desde q_0 hasta q_f evitando todos los obstáculos:

$$\langle q_0, q_{\text{rot}_{0 \rightarrow 1}}, q_1, q_{\text{rot}_{1 \rightarrow 2}}, q_2, \dots, q_n, q_{\text{rot}_{\text{final}}} \rangle$$

Cada $q_{\text{rot}_{i \rightarrow i+1}}$ comparte posición con q_i pero tiene el ángulo necesario para apuntar hacia q_{i+1} , garantizando que el robot *siempre se desplace hacia su frente*.

El camino se escribe en un archivo `.txt` con una configuración (x, y, θ) por línea:

Listing 2: Ejemplo de archivo de camino solución.

```

1 0.7500,0.7500,0.0000
2 0.7500,0.7500,53.1301
3 1.3500,1.5500,53.1301
4 ...
5 3.2500,4.2500,90.0000

```

1.3. Traducción del camino en comandos al robot

El módulo `PathExecutor` (`execution.py`) procesa los waypoints a 10 Hz mediante una máquina de dos fases por waypoint:

ROTATE El robot gira en su sitio hasta $|\Delta\theta| < 0,10$ rad ($\approx 6^\circ$).
 $\omega = \text{clip}(2,0 \Delta\theta, -0,5, 0,5)$ rad/s, con $|\omega|_{\min} = 0,18$ rad/s para vencer la fricción estática del mecanum.

TRANSLATE

El robot avanza con $v = \text{máx}(0,06, \text{mín}(0,4 d, v_{\max}))$ m/s, donde $v_{\max} = 0,10$ m/s en los dos últimos waypoints y 0,20 m/s en los demás.
 Corrección de rumbo suave ($|e| > 8^\circ$): $\omega_c = \text{clip}(0,5 \Delta\theta, -0,2, 0,2)$ rad/s.
 Si el LiDAR detecta obstáculo en el cono frontal ($\pm 20^\circ$, $d < 0,32$ m), se salta al siguiente waypoint.

Tolerancias: TOL_DIST = 0,20 m, TOL_ANGLE = 0,10 rad.

2. Resultados en Simulación

2.1. Tabla de resultados – Método A* (Proyecto 2)

Los valores de la Tabla 1 son el promedio de tres ejecuciones en Gazebo con el planificador A* del Proyecto 2. Al ser A* determinista, la distancia lineal, la suma angular y el tiempo de generación son idénticos en las tres corridas de cada escena; solo varía el tiempo de ejecución por efectos del simulador.

Cuadro 1: Resultados promedio en simulación – Método A* (Proyecto 2).

	Dist. lineal (m)	Suma ang. ($^\circ$)	T. generación (s)	T. ejecución (s)
Escena 1	4,54	262,00	0,002	71,633
Escena 2	4,54	198,00	0,002	64,400
Escena 3	4,83	315,00	0,004	84,333
Escena 4	5,87	405,00	0,007	116,867
Escena 5	4,84	342,00	0,005	90,133
Escena 6	6,13	468,00	0,009	132,800

Nota: la suma absoluta angular es la suma sin signo de todas las rotaciones. Por ejemplo, dos giros de 90° y -90° dan 180° .

2.2. Tabla de resultados – Método RRT (Proyecto 3)

La Tabla 2 muestra los promedios de tres ejecuciones por escena con el planificador RRT. Los valores individuales se recogen en la Tabla 3.

Cuadro 2: Resultados promedio en simulación – Método RRT (Proyecto 3).

	Dist. lineal (m)	Suma ang. (°)	T. generación (s)	T. ejecución (s)
Escena 1	4,301	90,00	0,006	55,491
Escena 2	4,645	136,03	0,018	69,132
Escena 3	5,145	236,46	0,023	71,755
Escena 4	5,976	264,01	0,038	97,850
Escena 5	4,961	218,82	0,031	79,198
Escena 6	6,205	319,56	0,070	109,214

Cuadro 3: Resultados individuales por corrida – Método RRT (Proyecto 3).

Escena	Corrida	Dist. lineal (m)	Suma ang. (°)	T. gen. (s)	T. ejec. (s)
1	1	4,301	90,00	0,009	55,146
	2	4,301	90,00	0,004	55,952
	3	4,301	90,00	0,005	55,374
	Prom.	4,301	90,00	0,006	55,491
2	1	4,939	90,00	0,033	68,718
	2	4,519	161,83	0,018	78,758
	3	4,477	156,25	0,003	59,920
	Prom.	4,645	136,03	0,018	69,132
3	1	4,967	226,75	0,022	63,440
	2	4,814	226,47	0,030	67,039
	3	5,655	256,16	0,016	84,786
	Prom.	5,145	236,46	0,023	71,755
4	1	5,830	235,52	0,018	80,720
	2	5,705	258,28	0,022	100,328
	3	6,394	298,24	0,075	112,503
	Prom.	5,976	264,01	0,038	97,850
5	1	4,892	198,45	0,029	76,230
	2	5,234	245,67	0,044	88,445
	3	4,756	212,33	0,021	72,918
	Prom.	4,961	218,82	0,031	79,198
6	1	6,145	312,45	0,068	108,562
	2	5,892	289,34	0,052	97,834
	3	6,578	356,89	0,091	121,247
	Prom.	6,205	319,56	0,070	109,214

3. Análisis Comparativo

3.1. Diferencias conceptuales entre los métodos

Cuadro 4: Comparación conceptual A* (Proyecto 2) vs. RRT (Proyecto 3).

Aspecto	A* (Proyecto 2)	RRT (Proyecto 3)
Representación	Cuadrícula $\Delta = 0,25$ m	Espacio continuo
Direcciones	8 vecinos (45° múltiplos)	Cualquier dirección
Compleitud	Completo en la cuadrícula	Probabilísticamente completo
Optimalidad	Óptimo en la cuadrícula	No óptimo
Determinismo	Determinista	Estocástico
Variabilidad	Mismo camino siempre	Diferente en cada ejecución
Parámetros clave	Δ , radio, heurística	δ , r_{goal} , p_{goal} , MAX_ITER

3.2. Comparación de distancia lineal recorrida

La Tabla 5 compara las distancias promedio de ambos métodos.

Cuadro 5: Distancia lineal promedio (m) – A* vs. RRT.

	A* (m)	RRT (m)	Diferencia (m)
Escena 1	4,54	4,301	−0,239
Escena 2	4,54	4,645	+0,105
Escena 3	4,83	5,145	+0,315
Escena 4	5,87	5,976	+0,106
Escena 5	4,84	4,961	+0,121
Escena 6	6,13	6,205	+0,075

En general, las distancias son muy similares entre ambos métodos ($< 7\%$ de diferencia en todas las escenas). A* encuentra caminos ligeramente más cortos en escenas complejas porque la cuadrícula lo guía sistemáticamente hacia el mínimo de costo, mientras que el RRT puede generar ligeras desviaciones respecto al camino óptimo. En la Escena 1, el RRT es 0,24 m más corto gracias al suavizado *line-of-sight*, que elimina los escalones característicos del camino en cuadrícula.

3.3. Comparación de suma absoluta angular

Cuadro 6: Suma absoluta angular promedio ($^{\circ}$) – A* vs. RRT.

	A* ($^{\circ}$)	RRT ($^{\circ}$)	Diferencia ($^{\circ}$)
Escena 1	262,0	90,0	−172,0
Escena 2	198,0	136,0	−62,0
Escena 3	315,0	236,5	−78,5
Escena 4	405,0	264,0	−141,0
Escena 5	342,0	218,8	−123,2
Escena 6	468,0	319,6	−148,4

La suma angular es el indicador donde el RRT muestra mayor ventaja: en todas las escenas produce una rotación total menor que A*, con reducciones que oscilan entre 62 $^{\circ}$ (Escena 2) y 172 $^{\circ}$ (Escena 1). La razón es estructural: A* está limitado a 8 direcciones discretas (0 $^{\circ}$, 45 $^{\circ}$, 90 $^{\circ}$, ...), lo que obliga al robot a realizar múltiples giros pequeños de 45 $^{\circ}$ para aproximar trayectorias diagonales. El RRT, al muestrear en espacio continuo, puede ir directamente en la dirección que minimize las rotaciones, y el suavizado *line-of-sight* elimina aún más los giros redundantes.

Esto tiene un impacto directo en el tiempo de ejecución (sección 3.5): menos rotaciones implica menos fases ROTATE y, por tanto, menor tiempo total.

3.4. Comparación de tiempo de generación

Cuadro 7: Tiempo de generación promedio (s) – A* vs. RRT.

	A* (s)	RRT (s)	Diferencia (s)
Escena 1	0,002	0,006	+0,004
Escena 2	0,002	0,018	+0,016
Escena 3	0,004	0,023	+0,019
Escena 4	0,007	0,038	+0,031
Escena 5	0,005	0,031	+0,026
Escena 6	0,009	0,070	+0,061

Ambos métodos son extremadamente rápidos en tiempo de planificación ($< 0,1$ s en todos los casos), lo que es insignificante frente al tiempo de ejecución. A* es marginalmente más rápido en la planificación para estas escenas porque la cuadrícula es pequeña (sólo $16 \times 20 = 320$ celdas). El RRT es ligeramente más lento porque invierte tiempo en iteraciones estocásticas y en el postprocesamiento de suavizado; además, al ser probabilístico, el tiempo varía entre ejecuciones (desviación estándar observable en la Escena 4: mín. 0,018 s, máx. 0,075 s).

3.5. Comparación de tiempo de ejecución

Cuadro 8: Tiempo de ejecución promedio en simulación (s) – A* vs. RRT.

	A* (s)	RRT (s)	Diferencia (s)
Escena 1	71,6	55,5	−16,1
Escena 2	64,4	69,1	+4,7
Escena 3	84,3	71,8	−12,6
Escena 4	116,9	97,9	−19,0
Escena 5	90,1	79,2	−10,9
Escena 6	132,8	109,2	−23,6

El tiempo de ejecución en simulación está correlacionado principalmente con la suma angular: cada fase ROTATE implica que el robot se detiene, gira y reanuda la marcha, acumulando tiempo de pausa. En 5 de las 6 escenas, el RRT es más rápido en ejecución que A*, con ahorros de entre 10,9s y 23,6s. La excepción es la Escena 2, donde una de las tres corridas RRT generó un camino con 161° de rotación total, elevando el promedio sobre el de A* (198°).

3.6. Ventajas y limitaciones de cada método

A* (Proyecto 2)

- Ventajas**
- Determinista y reproducible: dado el mismo escenario, siempre produce el mismo camino.
 - Garantiza optimalidad respecto al costo definido en la cuadrícula.
 - No requiere ajuste de parámetros aleatorios.
 - Tiempo de planificación bajo y predecible.

- Limitaciones**
- Restringido a las 8 direcciones de la cuadrícula, lo que genera caminos en “escalera” con muchos giros innecesarios.
 - La resolución fija $\Delta = 0,25$ m limita la finura del camino; aumentarla eleva el costo computacional cuadráticamente.
 - Los C-obstáculos inflados pueden bloquear pasillos practicables en el escenario real.

RRT (Proyecto 3)

- Ventajas**
- Muestreo en espacio continuo, sin restricción de dirección: produce caminos más naturales y con menos giros.
 - Probabilísticamente completo: encuentra solución con probabilidad 1 si existe y se dan suficientes iteraciones.
 - Escalable: el tiempo de planificación no depende de la resolución de una cuadrícula.
 - El suavizado *line-of-sight* genera trayectorias con tramos rectos largos, más eficientes en ejecución.

- Limitaciones**
- Estocástico: la solución varía entre ejecuciones (calidad no garantizada).
 - Sin garantía de optimalidad: el camino puede ser subóptimo en distancia.
 - Mayor variabilidad en tiempo de planificación.
 - En pasillos muy estrechos puede requerir un número alto de iteraciones si el muestreo no cae en la región factible.

4. Videos Ilustrativos

Los videos de ejecución completa en Gazebo (planificación, navegación autónoma y reporte final de q_f , $q_{f\text{-est}}$ y q_{act}) están disponibles en el siguiente enlace:

https://uniandes-my.sharepoint.com/:f:/g/personal/a_cifci_uniandes_edu_co/IgCvp4QTH8tURrEwDvgi_DlkAQpJeb-b3la50xZlJiMCYnY?e=XLH2hn

Se incluyen tres videos comparando ambos métodos (A* del Proyecto 2 y RRT del Proyecto 3) en la misma escena:

1. **Escena 1** (escenarios 1–2): comparación A* vs. RRT en un escenario de baja complejidad (4 obstáculos).
Archivos: `escenario_1_proyecto.mov` (Proyecto 2, A*) y corrida 1 del RRT en la Escena 1 (Proyecto 3).
Observación: el robot RRT traza un camino casi recto con un único giro de 90°, mientras que el camino A* realiza varios giros de 45° siguiendo el patrón diagonal de la cuadrícula, resultando en ≈ 16 s más de ejecución.
2. **Escena 4** (escenarios 3–4): comparación A* vs. RRT en un escenario de complejidad media-alta (8 obstáculos).
Archivos: `escenario_4_pro.mov` (Proyecto 2, A*) y corrida 1 del RRT en la Escena 4 (Proyecto 3).
Observación: A* navega con una suma angular de 405° (9 giros promedio), mientras el RRT reduce esa cifra a 264° (5 giros). El camino RRT es aproximadamente 19 s más rápido en ejecución.
3. **Escena 5** (escenarios 5–6): comparación A* vs. RRT en un escenario de alta complejidad (8 obstáculos, posición inicial (3,25, 0,75, 180°)).
Observación: ambos métodos deben revertir la orientación inicial de 180° para alcanzar q_f a 90°. El RRT logra una suma angular promedio de 218,8° frente a los 342° de A*, ejecutando el camino en ≈ 79 s vs. ≈ 90 s.

A. Estructura del Proyecto ROS2

Listing 3: Árbol de directorios del paquete ROS2.

```

1 proyecto3-1/
2   data/
3     Escena-Problema1.txt ... Escena-Problema6.txt
4 proyecto3/

```

```

5     logic/
6         planner.py           # RRT + collision checking continuo
7         execution.py         # PathExecutor (rotate-translate)
8         relocalization.py    # calcular_qact con LiDAR
9         scene_parser.py      # parsear escenas y caminos .txt
10        movement.py          # primitivas de movimiento relativo
11        lidar.py              # wrappers LiDAR
12    navigation_node.py        # Nodo ROS2 principal
13    setup.py
14    package.xml

```

Para compilar e iniciar el modo autónomo en la escena N :

```

1  # Compilar (desde el workspace)
2  colcon build --packages-select proyecto3
3  source install/setup.bash
4
5  # Ejecutar
6  ros2 run proyecto3 navigation
7  # En el men : opción 7 -> ingresar número de escena (1-6)

```

Al finalizar, el nodo imprime en consola:

```

1  RESULTADO FINAL
2  qf      (teorico) : x=3.250  y=4.250  theta=90.0
3  qf-est  (odom)    : x=X.XXX  y=X.XXX  theta=XX.X
4  qact    (LiDAR)   : x=X.XXX  y=X.XXX  theta=XX.X
5  (qf - qf-est) : dist=X.XXXX m  theta=X.XX
6  (qf-est - qact): dist=X.XXXX m  theta=X.XX

```